

@April 27, 2026

Summary

Action Items

- ☐ Create sequence diagrams showing workflows for new bugs vs bugs with incremental changes ▶ ▶
- ☐ Add RAG implementation to the architecture block diagrams ▶
- ☐ Redesign UI homepage to display auto-discovered bug list instead of manual entry ▶ ▶ ▶
- ☐ Implement dashboard with bug status indicators and critical bug counts ▶
- ☐ Add filtering options for projects, source systems, and bug status ▶ ▶
- ☐ Implement pagination for bug listings ▶ ▶
- ☐ Add similarity scoring indicators for related issues showing percentage match ▶ ▶
- ☐ Enhance LLM matching to include bug title, description, logs, timestamps, customer impact, troubleshooting steps, and comments ▶ ▶ ▶ ▶ ▶
- ☐ Create token settings page allowing users to configure API credentials, base URLs, and port numbers for each bug tracking system ▶ ▶
- ☐ Develop detailed use case diagrams and sequence diagrams breaking down the system into smaller components ▶ ▶
- ☐ Create implementation plan showing how work will be distributed among team members ▶
- ☐ Add Knowledge Base support portal to external sources ▶
- ☐ Upload all meeting recordings and documents to GitHub and share link ▶
- ☐ Submit checkpoint review materials using this recording ▶ ▶

Redesigned Architecture Overview

- Team presented updated High-Level Design based on previous feedback ▶
- System starts with engineer submitting bug ID/issue, which is authenticated using JWT tokens stored securely in token vault ▶
- Token vault also stores API keys and credentials for external systems like Jira, GitHub, and Bugzilla ▶
- FastAPI serves as entry layer, validates JWT tokens, and pushes requests to Kafka topic for fault tolerance and scalability ▶ ▶

Agent Pipeline Architecture

- Task orchestrator acts as central coordinator managing all agent communication ▶
- Agents are completely isolated from each other and communicate only through orchestrator ▶
- **Phase 1:** Context agent runs alone, fetches full bug details from source system, retrieves linked customer case notes, and extracts keywords for downstream agents ▶
- **Phase 2:** Cross-system fetch agent and enrichment agent run in parallel after phase 1 completes ▶
- Cross-system fetch agent searches Bugzilla for related bug reports and GitHub for related issues using keywords from context agent ▶
- Enrichment agent extracts keywords, builds SQL queries, searches Confluence in read-only mode, and returns KB articles, runbooks, and past incident reports ▶
- AI synthesis agent combines outputs from all three agents and sends to LLM ▶
- LLM returns structured JSON with suggested severity, probable root cause, recommended next steps, and source references ▶

External System Integration

- Connector registry pattern replaces hardcoded if-statements for extensibility ▶
▶

- Each external system (Jira, Bugzilla, GitHub) has individual adapter class in registry ▶
- Adding new systems only requires updating connector registry without changing pipeline logic ▶
- Redis layer with 5-minute TTL cache prevents redundant API calls for repeated queries while maintaining data sovereignty ▶

UI Design Feedback and Required Changes

- Current design requires manual bug ID entry, which needs to be eliminated ▶ ▶
- Tool should auto-discover all bugs requiring triage and display them in list view ▶ ▶
- Implement collapsible/expandable rows showing triage details without requiring navigation ▶
- Add search functionality within list view and multiple filter options for projects and source systems ▶
- Include status indicators showing severity, priority, and whether bug is closed ▶
- Current history tab shows right concept but needs complete listing with pagination, not just recent items ▶ ▶
- Create tree view for correlated bugs using one bug system ID as primary key with others as child nodes ▶

Bug Correlation and Matching Requirements

- Related issues must show similarity scores indicating percentage match confidence ▶
- Implement LLM-based matching instead of keyword search alone ▶ ▶
- Matching should consider multiple fields: bug title, keywords, tags, status whiteboard, customer impact statement, severity, serial numbers, timestamps, descriptions, troubleshooting steps, maintenance changes, workarounds, log information, crash dumps, performance data, and all engineer comments ▶ ▶ ▶
▶ ▶ ▶

- Similarity scoring tiers: >95% excellent match, >90% good match ▶
- Some bugs have direct references (like CFI customer-found issues) that should be utilized when available ▶

Data Management and Caching Strategy

- System needs mechanism to detect and fetch incremental changes from dynamic bug tracking systems ▶ ▶
- Should fetch latest data from source systems without storing everything in own database ▶
- Temporary caching in PostgreSQL allowed during pipeline execution for specific bug and associated bugs ▶ ▶ ▶
- Cache must be flushed after processing completes ▶
- Should not create data bottlenecks by storing all bug details permanently ▶

UI Result Dashboard

- Four main sections displayed after pipeline completes ▶
- Bug context panel shows core details (title, priority, status, component, assignee) from context agent ▶
- Related issues panel displays similar bugs from other systems with direct links to original sources ▶
- Linked context panel shows KB articles and runbooks from enrichment agent ▶
- AI summary panel consolidates all information showing suggested severity, root causes, and recommended steps ▶
- Pipeline progress visible through status indicators and progress bars ▶ ▶

Token Configuration Requirements

- Need settings page for users to configure API tokens for each bug tracking system ▶ ▶
- Must allow multiple URLs per system (e.g., different Jira projects) ▶

- Users should provide base URL including port number and API token for each ▶
- Backend should support JSON configuration for all setting details ▶

External Sources

- Current external systems: Jira, Bugzilla, GitHub, Confluence, Customer Portal ▶
- Approved to add Knowledge Base support portal for documented workarounds and known issues ▶ ▶